

Revision — 2.1.1 Thinking Abstractly

OCR H446 — one-page revision summary

Must know

- **Abstraction** = removing or hiding unnecessary detail so only information **relevant to the problem** remains, giving a simpler model.
- Two types: **representational abstraction** (simplify *one specific* thing → a model of that problem, e.g. a road network → a graph of nodes and weighted edges) and **generalisation / categorisation** (group *many* things by shared characteristics → a reusable model, e.g. one `Account` type for savings and current accounts).
- The skill is choosing which detail to **keep** and which to **discard**, and **justifying** why the discarded detail is irrelevant *to this problem*.
- A good model (tube map, weather grid, chess state) keeps only what matters for the task — e.g. a tube map discards true geography but keeps lines, station order and interchanges.
- **Abstraction ≠ decomposition**. Abstraction *removes/hides* detail; decomposition *breaks* a problem into sub-problems. Never blur them.
- Removing irrelevant detail makes a system **simpler, faster to build and easier to maintain** while still solving the problem.
- A generalised model makes a program **easier to extend** — new cases reuse/extend the existing model instead of new code, cutting effort and bugs.

Key terms

Term	Definition
Abstraction	Removing/hiding detail irrelevant to the problem to give a simpler model
Representational abstraction	Simplifying one specific thing into a model of that problem
Generalisation	Grouping many things by shared characteristics into a reusable model
Model	Simplified representation of a real-world thing or system
Decomposition	Breaking a problem into smaller sub-problems (a <i>different</i> skill)
Node	A point (e.g. junction/place) in a graph abstraction
Weighted edge	A connection carrying a cost such as distance or time

Worked example / how to — apply abstraction to a hospital appointments system

To model a "patient" for appointment scheduling:

- **Keep:** patient ID/name, date of birth, contact number, GP/clinic, appointment type and duration — each is needed to schedule and contact the patient.
- **Discard:** eye colour, favourite food, home décor.

- **Justify the discard:** none of these affect scheduling or contacting the patient, so including them adds complexity without value.
- **Conclusion:** keeping only relevant detail makes the system simpler, quicker to build and easier to maintain while still solving the appointments problem.

Exam tips

- **Apply, don't define** — name the scenario's specific data to **keep** and detail to **discard**, and **justify** each discard (the justification often carries the mark).
- Distinguish **representational abstraction** vs **generalisation** with a valid example of each.
- Don't confuse **abstraction** (removes detail) with **decomposition** (splits the problem) — a very common trap.
- Tie your answer back to a benefit: simpler, faster to build, easier to extend/maintain.